



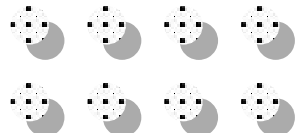
Presentación de Jueguito

POO en C++ | V0.1.1 | ***Equipo: Los Cholos***

09/28/26

ÍNDICE

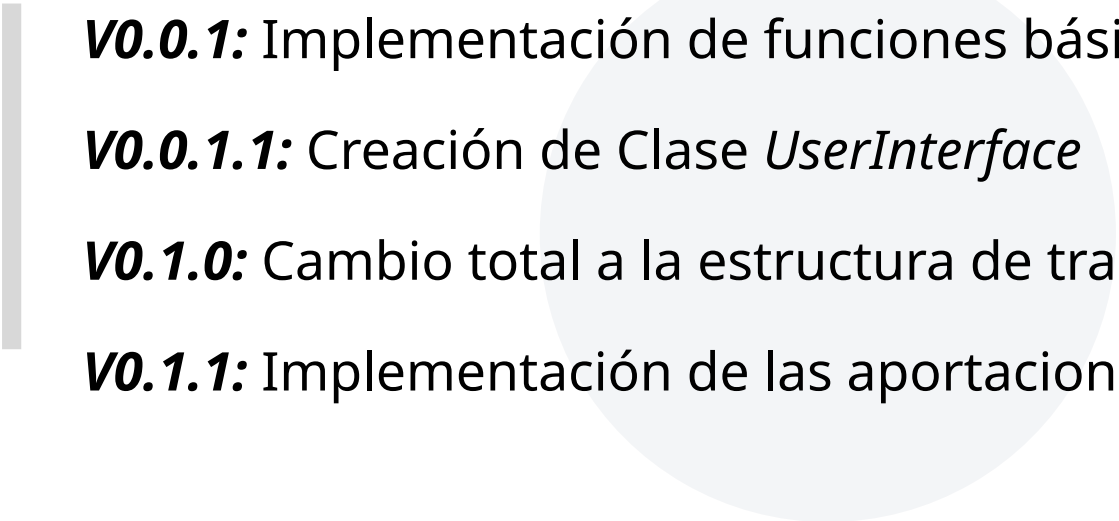
- Arquitectura o Diseño
- Código e Implementación
- Testeo
- Documentación Final





ARQUITECTURA

Para el diseño general del proyecto, el Arquitecto principal fue Derek Salcido González. El proceso de diseño se dividió en las siguientes fases según los proyectos individuales:



V0.0.1: Implementación de funciones básicas

V0.0.1.1: Creación de Clase *UserInterface*

V0.1.0: Cambio total a la estructura de trabajo

V0.1.1: Implementación de las aportaciones de Luis y Javier



Versión 0.0.1

01

Crear función
void Output()

03

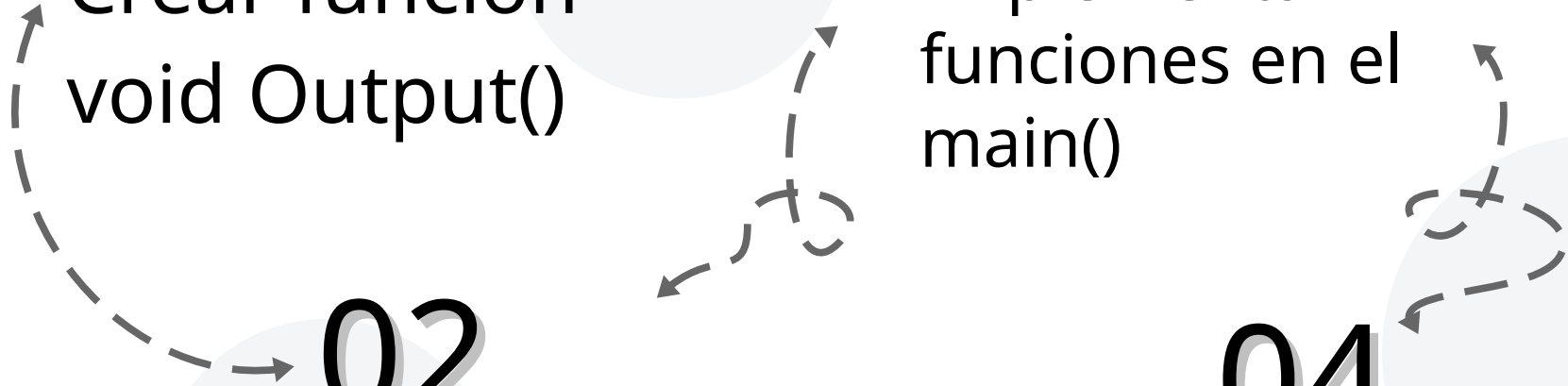
Implementar
funciones en el
main()

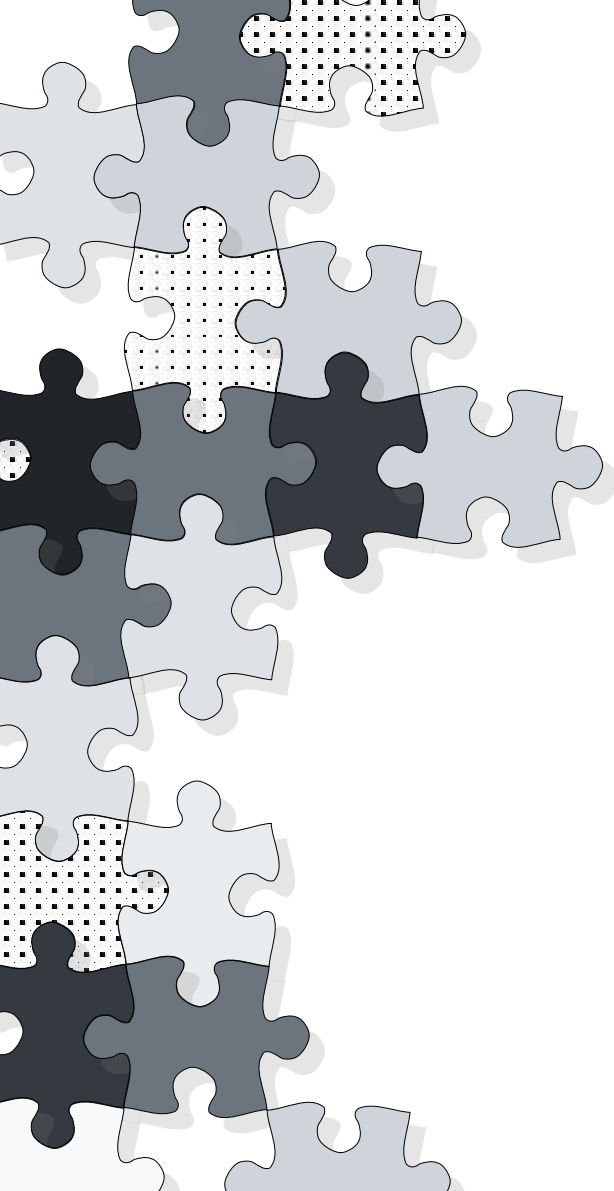
02

Crear función int
Input()

04

Programar uso
práctico en el main()

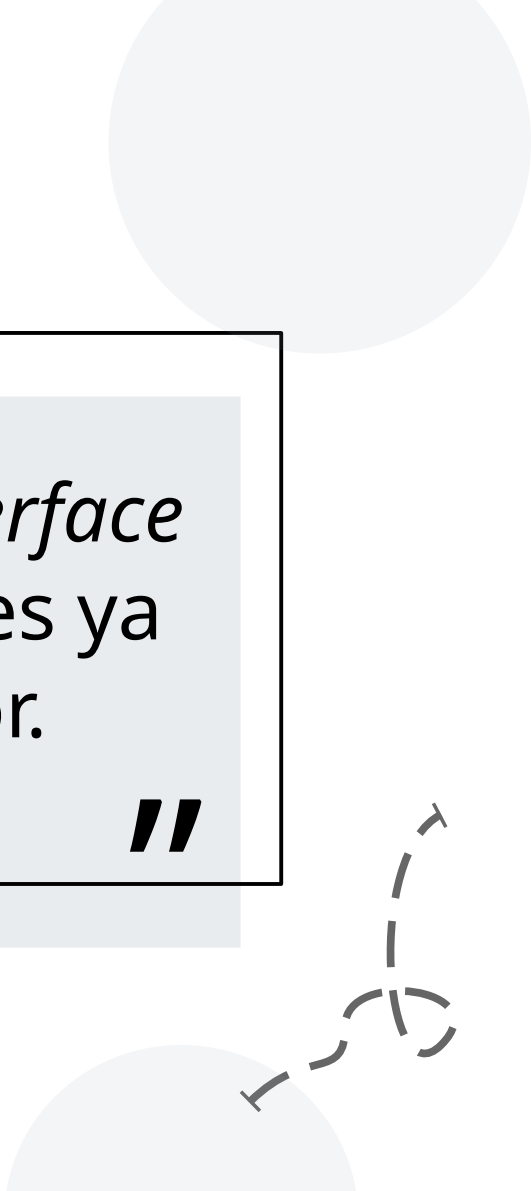




Versión 0.0.1.1

//
Crear clase *UserInterface*
y pasar las funciones ya
creadas a su interior.

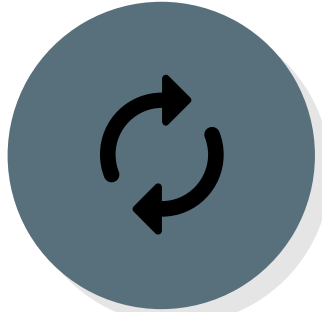
//



Versión 0.1.0 | Cambios



Código
desordenado e
incomprensible



División del código
en archivos .hpp
y .cpp



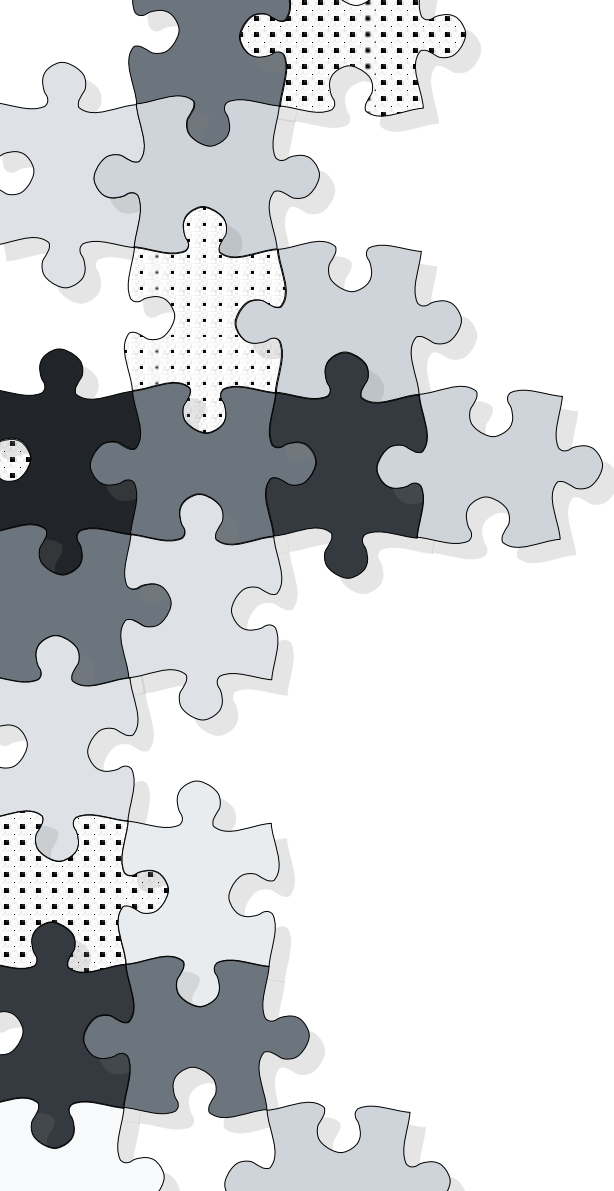
Implementación
para mejorar la
comprensión

Versión 0.1.0

Se pasó la clase *UserInterface* hacia los archivos *Class_UserInterface .cpp/.hpp*.

Se implementó al *jueguito.cpp* y se creó menú de selección de modos de juego:

1: Modo Historia | 2: Modo Batalla

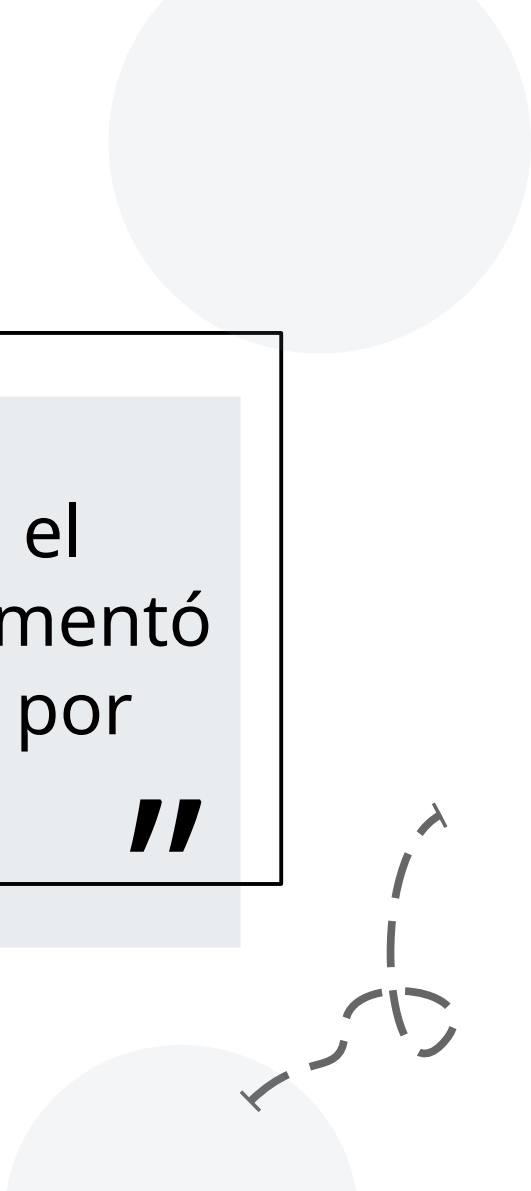


Versión 0.1.1

//

Al ser el Modo Batalla el más pedido, se implementó al proyecto, aportado por Luis y Javier.

//



Versión 0.1.1

01

Obtener las aportaciones dadas por Luis y Javier

03

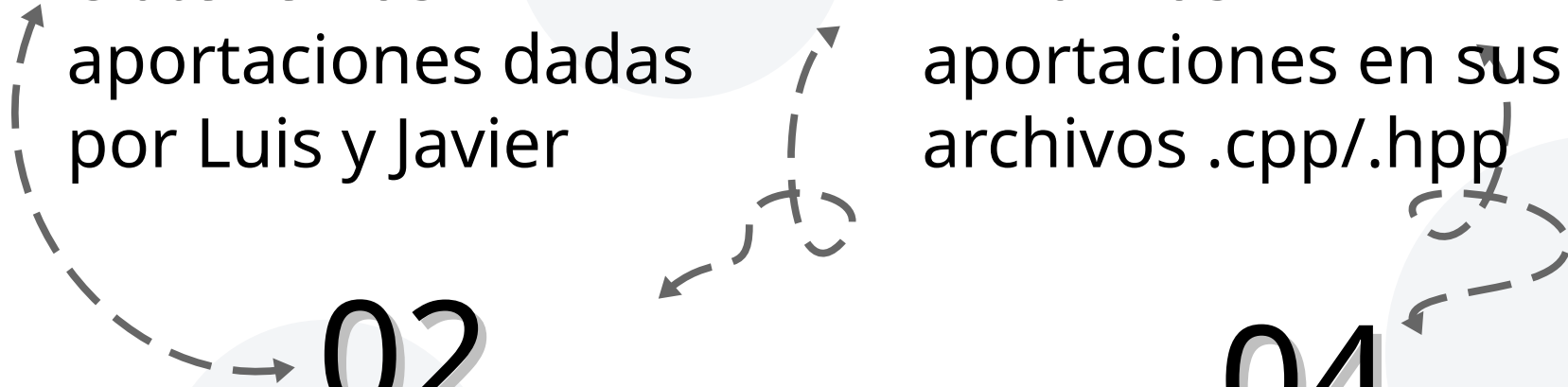
Dividir las aportaciones en sus archivos .cpp/.hpp

02

Reorganizar el código fuente

04

Implementar su uso en jueguito.cpp



CÓDIGO E IMPLEMENTACIÓN

Para la verdadera programación del proyecto, los programadores principales fueron Luis, Javier, Omar y Derek. Se dividió el trabajo en estas siguientes fases:

V0.0.1: Erick, Omar y Derek

V0.0.1.1: Derek

V0.1.0: Luis, Javier y Derek

V0.1.1: Luis, Javier, Omar y Derek

CÓDIGO V0.0.1

Primera implementación

```
5 + void Output()  
6 + {  
7 +     string texto = "Bienvenido al juego.\n"  
8 +         "Elige una opcion para continuar.\n";  
9 +  
10 +     cout << texto;  
11 + }
```

Actualización

```
9         void Output(string texto)  
10        {  
11            cout << texto;  
12        }  
13    };
```

Implementación: Omar

CÓDIGO V0.0.1

```
25      /* Metodo para recibir las opciones del jugador */
26  ✓ void Input(int InferiorLimit, int SuperiorLimit, const std::string& Question){
27      do{
28          /* Se imprime el texto de la pregunta con una sangria */
29          std::cout << "    " << Question << std::endl << "    ";
30
31          /* Se imprimen/muestran las opciones a escoger en forma de matriz lineal */
32          for(int i=InferiorLimit; i<=SuperiorLimit; i++){
33              std::cout << " | " << i << " ";
34          }
35          std::cout << " | -> ";
36
37          /* Se pide la opcion a escoger */
38          std::cin >> Option;
39
40          /* Si el usuario mete alguna letra o numero indeseado, se ejecutara el "if" para evitar errores futuros */
41          if(std::cin.fail() || std::cin.peek() != '\n'){
42              std::cin.clear();
43              std::cin.ignore(100 /* Rango de limpieza de 100 caracteres */, '\n');
44              Option = InferiorLimit-1;
45          }
46
47          /* Se repetira el ciclo hasta que se introduzca alguno de los valores del rango */
48          }while(Option < InferiorLimit || Option > SuperiorLimit);
49      }
```

Implementación: Erick

CÓDIGO V0.0.1.1

```
4  class UserInterface{
```

```
5      private:
6          /* Variable encargada de almacenar las opciones del usuario/jugador.
7             Es privada para que no se pueda modificar de ninguna manera fuera de la clase en si */
8          int Option;
9      public:
10
11          /* Constructor que inicializa la variable 'Option' a 0 por default */
12          UserInterface(){
13              Option = 0;
14          }
```

Implementación: Omar

CÓDIGO V0.0.1.1

```
51      /* Metodo 'get' para obtener la opcion del jugador */
52      int GetOption(){
53          return Option;
54      }
55  };
56
57  ✓ int main(){
58
59      /* Se crea la interfaz de usuario UI */
60      UserInterface UI;
61      std::string texto = "Bienvenido al juego.";
62
63      /* Codigo de prueba. Los testers pueden jugar con las funciones de UI como quieran para encontrar errores */
64      UI.Output(texto);
65      UI.Input(1, 3, "Elije una opcion para continuar: ");
66      std::cout << UI.GetOption() << std::endl; /* Ejemplo para poner a prueba 'UI.GetOption()' */
67      UI.Output("Has escogido la opcion!");
68      UI.Input(-1, 2, "Que opcion deseas escojer?");
69
70      return 0;
71  }
```

Implementación: Omar y Derek

NUEVO ORDEN EN V0.1.0

Repositorio_clase_poo_Equipo-los-cholos_videojuego_ / individuales /

 **javiermora3269-ai** Add files via upload

Name	Last commit message
..	
Cristian	Create Proyecto de Cristian
Derek	Fixed infinite loop inside main.cpp in Proyecto de Derek
Erick	Implement Criatura class for battle simulation
Javi	Implementacion_proyecto_Videojuego
Luis	Add files via upload
Omar	Add files via upload

Implementación: Todos

NUEVO ORDEN EN V0.1.0

Repositorio_clase_poo_Equipo-los-cholos_videojuego_ / individuales / Luis /

 javiermora3269-ai Add files via upload	
Name	Last commit message
 ..	
 Proyecto de Luis	Update Proyecto de Luis
 proyecto_luis_implementacion_javi.cpp	Add files via upload

Implementación: Luis y Javier

CÓDIGO V0.1.0

```
14 // -----  
15 // Estructura auxiliar para definir los ataques de cada criatura  
16 // -----  
17 > struct Ataque ...  
22 };  
23  
24 // -----  
25 // DEFINICION DE LA CLASE: Criatura  
26 // -----  
27 > class Criatura ...  
301 };
```

```
305 // -----  
306 // FUNCIONES DE FABRICA Y SELECCION  
307 // -----  
308  
309 > void mostrarMenuCriaturas() ...  
316 }  
317  
318 > Criatura crearCriaturaPorOpcion(int opcion) ...  
370 }  
371  
372 //  
373 // FUNCION PARA EL TURNO DE LA IA (1 JUGADOR)  
374 // Aportacion Javier Mora Gutierrez  
375 > void ejecutarTurnoIA(Criatura &oponente, Criatura &jugador, int dificultad) ...  
439 }  
440  
441 // -----  
442 // FUNCION PRINCIPAL (main)  
443 // -----  
444 > int main() ...  
689 }
```

Implementación: Luis y Javier

CÓDIGO V0.1.0

```
444 int main()
445 {
446     srand(time(0));
447
448     // -----
449     // SECCION 1: Bienvenida y Seleccion de Modo de Juego
450     // -----
451     cout << "===== " << endl;
452     cout << "    Bienvenido al Jueguito" << endl;
453     cout << "===== " << endl
454         << endl;
455     int dificultad = 1; //Variable para escoger la dificultad Javier Mora Gutierrez
456     int modoJuego = 1;
457     cout << "SELECCIONA EL MODO DE JUEGO:" << endl;
458     cout << "1. 1 Jugador (Player 1 vs IA)" << endl;
459     cout << "2. 2 Jugadores (Player 1 vs Player 2)" << endl;
460     cout << "Elige una opcion: ";
461     cin >> modoJuego;
462
463     //Seleccion de dificultad Javier Mora Gutierrez
464     if (modoJuego == 1) {
465         cout << "\nSELECCIONA LA DIFICULTAD DEL BOT:" << endl;
466         cout << "1. Facil" << endl;
467         cout << "2. Medio" << endl;
468         cout << "3. Dificil" << endl;
469         cout << "Elige la dificultad: ";
470         cin >> dificultad;
471         if (dificultad < 1 || dificultad > 3) dificultad = 1;
472     }
```

Implementación: Luis y Javier

CÓDIGO V0.1.0

```
477 // -----
478 // SECCION 2: Seleccion de Personajes
479 // -----
480 if (modoJuego == 2)
481 {
482     cout << "\n--- PLAYER 1: SELECCIONA TU CRIATURA ---" << endl;
483 }
484 else
485 {
486     cout << "\n--- SELECCIONA TU CRIATURA ---" << endl;
487 }
488 mostrarMenuCriaturas();
489 cout << "Elige tu opcion: ";
490 cin >> opcionJugador;
491
492 Criatura jugador = crearCriaturaPorOpcion(opcionJugador);
493
494 if (modoJuego == 2)
495 {
496     cout << "\n--- PLAYER 2: SELECCIONA TU CRIATURA ---" << endl;
497     mostrarMenuCriaturas();
498     cout << "Elige tu opcion: ";
499     cin >> opcionOponente;
500 }
501 else
502 {
503     // Seleccion aleatoria para el oponente en modo 1 Jugador
504     opcionOponente = (rand() % 5) + 1;
505     cout << "\nSE HA SELECCIONADO EL OPONENTE DE MANERA ALEATORIA." << endl;
506 }
507
508 Criatura oponente = crearCriaturaPorOpcion(opcionOponente);
```

Implementación: Luis

CÓDIGO V0.1.0

```
529     cout << "===== " << endl;
530     cout << " EMPIEZA EL COMBATE: " << jugador.obtenerNombre()
531         << " VS " << oponente.obtenerNombre() << endl;
532     cout << "===== " << endl;
533
534     while (jugador.estaViva() && oponente.estaViva())
535     {
536         cout << "\n===== " << endl;
537         cout << "             ROUND " << ronda << endl;
538         cout << "===== " << endl;
539         jugador.mostrar();
540         oponente.mostrar();
541
542         // -----
543         // TURNO DEL JUGADOR 1
544         // -----
545         bool accionTurnoValida = false;
546
547         // VERIFICACIÓN DE SEGURIDAD: Si está sin energía y sin usos restantes
548         //Javier Mora Gutierrez
549         if (jugador.estaBloqueada())
550         {
551             cout << "\n  [!] ¡Te has quedado sin energía, descansos y curaciones!" << endl;
552             cout << "  [!] Pasas tu turno de emergencia y recuperas 15 pts de energía." << endl;
553             jugador.setEnergia(jugador.obtenerEnergia() + 15);
554             accionTurnoValida = true; // Activa la bandera para no entrar al menú
555         }
556     }
```

Implementación: Luis

CÓDIGO V0.1.0

```
557 while (!accionTurnoValida && jugador.estaViva())
558 {
559     cout << "\nQue accion deseas realizar, " << jugador.obtenerNombre() << "?" << endl;
560     cout << "1. Atacar" << endl;
561     cout << "2. Descansar (Recupera energia)" << endl;
562     cout << "3. Curar (Recupera vida)" << endl;
563     cout << "Elige tu accion: ";
564
565     int eleccionAccion = 0;
566     cin >> eleccionAccion;
567
568     if (eleccionAccion == 1)
569     {
570         jugador.mostrarAtaques();
571         cout << "Elige un ataque: ";
572         int numAtaque = 0;
573         cin >> numAtaque;
574
575         accionTurnoValida = jugador.ejecutarAtaque(numAtaque - 1, oponente);
576     }
577     else if (eleccionAccion == 2)
578     {
579         // Invocacion del metodo descansar() (Retorna false si excede limite)
580         accionTurnoValida = jugador.descansar();
581     }
582     else if (eleccionAccion == 3)
583     {
584         // Invocacion del metodo curar() (Retorna false si excede limite)
585         accionTurnoValida = jugador.curar();
586     }
587     else
588     {
589         cout << "Opcion no valida. Intenta de nuevo." << endl;
590     }
591 }
```

Implementación: Luis

CÓDIGO V0.1.0

```
593         // Verificar si el oponente murio tras el ataque del jugador 1
594         if (!oponente.estaViva())
595         {
596             cout << "\n"
597                 << oponente.obtenerNombre() << " ha sido derrotado" << endl;
598             break;
599         }
```

Implementación: Luis

CÓDIGO V0.1.0

```
644         else
645         {
646             // TURNO DEL Oponente (IA según la dificultad elegida)
647             //Javier Mora Gutierrez
648             ejecutarTurnoIA(oponente, jugador, dificultad);
649         }
650
651         // Verificar si el jugador murió tras el ataque del oponente / Player 2
652         if (!jugador.estaViva())
653         {
654             cout << "\n"
655                 << jugador.obtenerNombre() << " ha sido derrotado" << endl;
656             break;
657         }
658
659         ronda++;
660     }
```

Implementación: Luis

CÓDIGO V0.1.0

```
662 // -----
663 // SECCION 5: Resultado Final del Combate
664 // -----
665 cout << "\n===== " << endl;
666 cout << "          FIN DEL COMBATE" << endl;
667 cout << "===== " << endl;
668
669 if (jugador.estaViva() && !oponente.estaViva())
670 {
671     cout << "VICTORIA " << jugador.obtenerNombre() << " es el ganador." << endl;
672 }
673 else if (!jugador.estaViva() && oponente.estaViva())
674 {
675     cout << "DERROTA " << oponente.obtenerNombre() << " te ha vencido." << endl;
676 }
677 else
678 {
679     cout << "EMPATE. Ambas criaturas cayeron en combate." << endl;
680 }
681
682 cout << "\nEstado Final:" << endl;
683 jugador.mostrar();
684 oponente.mostrar();
685
686 cout << "\nFIN DEL JUEGO" << endl;
```

Implementación: Luis

CÓDIGO V0.1.0

```
375 void ejecutarTurnoIA(Criatura &oponente, Criatura &jugador, int dificultad)
376 {
377     cout << "\n--- TURNO DEL Oponente [BOT] (" << oponente.obtenerNombre() << ") ---" << endl;
378
379     // 1. FACIL: Decisiones al azar
380     if (dificultad == 1)
381     {
382         if (rand() % 2 == 0)
383         {
384             if (!oponente.ejecutarAtaque(rand() % 4, jugador))
385                 oponente.descansar();
386         }
387         else
388         {
389             if (!oponente.curar())
390                 oponente.descansar();
391         }
392         return;
393     }
```

Implementación: Javier

CÓDIGO V0.1.0

```
408 // 3. DIFÍCIL: Se cura en estado crítico o selecciona el ataque con mayor daño
409 if (dificultad == 3)
410 {
411     if (oponente.obtenerVida() < (oponente.obtenerVidaMaxima() * 0.25))
412     {
413         if (oponente.curar())
414             return;
415     }
416
417     int mejorAtaque = -1;
418     int maxDanio = -1;
419
420     for (int i = 0; i < 4; i++)
421     {
422         Ataque a = oponente.obtenerAtaque(i);
423         if (oponente.obtenerEnergia() >= a.costoEnergia && a.danio > maxDanio)
424         {
425             maxDanio = a.danio;
426             mejorAtaque = i;
427         }
428     }
429
430     if (mejorAtaque != -1)
431     {
432         oponente.ejecutarAtaque(mejorAtaque, jugador);
433     }
434     else
435     {
436         oponente.descansar();
437     }
438 }
```

Implementación: Luis y Javier

CÓDIGO V0.1.0

```
22  /* MAIN / JUEGO PRINCIPAL */
23  ✓ int main(){
24
25      /* Se crea la interfaz de usuario UI */
26      std::string texto = "Bienvenido al juego.";
27      bool Termino = false; //ESTA VARIABLE SOLO EXISTE POR AHORA PARA NO TENER UN BUCLE INFINITO INESCESARIO DENTRO DEL PROYECTO
28
29      // AQUI ES DONDE COMIENZA EL JUEGO DE VERDAD
30      do{
31          UI.Output("== BIENVENIDO A JUEGUITO ==", 25, 25);
32          DisplayMenu(1);
33          UI.Input(0, 2, "");
34          switch(UI.GetOption()){
```

Implementación: Derek

CÓDIGO V0.1.0

```
35         case 0:
36             Termino = true;
37             break;
38         case 1:
39
40             // CREAR FUNCION DONDE SE DESARROLLE LA TOTALIDAD DE LA HISTORIA
41             std::cout << "Estas dentro del modo historia" << std::endl;
42             Termino = true;
43             break;
44
45         case 2:
46
47             // CREAR FUNCIONES DONDE SE DESARROLLEN CADA UNO DE LOS MODOS DE JUEGO
48             DisplayMenu(2);
49             UI.Input(1, 3, "");
50
51             // EJEMPLO DE IMPLEMENTACION DE LOS MODOS DE JUEGO
52             switch(UI.GetOption()){
53                 case 1:
54
55                     std::cout << "Estas dentro del modo PvP" << std::endl;
56                     Termino = true;
57                     break;
58
59                 case 2:
60
61                     std::cout << "Estas dentro del modo PvE" << std::endl;
62                     Termino = true;
63                     break;
64
65                 default: break;
66             }
```

Implementación: Derek

IMPLEMENTACIÓN V0.1.1

Gracias al arduo trabajo de programadores que hicieron Luis y Javier, se pudieron agregar las mecánicas de su proyecto individual al proyecto grupal, pero no sin antes modificar el orden de su código.

Para lograr su implementación, no bastaba con Copiar y Pegar; se necesitaba dividir todo el código en bloques bien definidos para poder combinarlo con el proyecto real. Esto se logró mediante la división del código en archivos .cpp y .hpp.

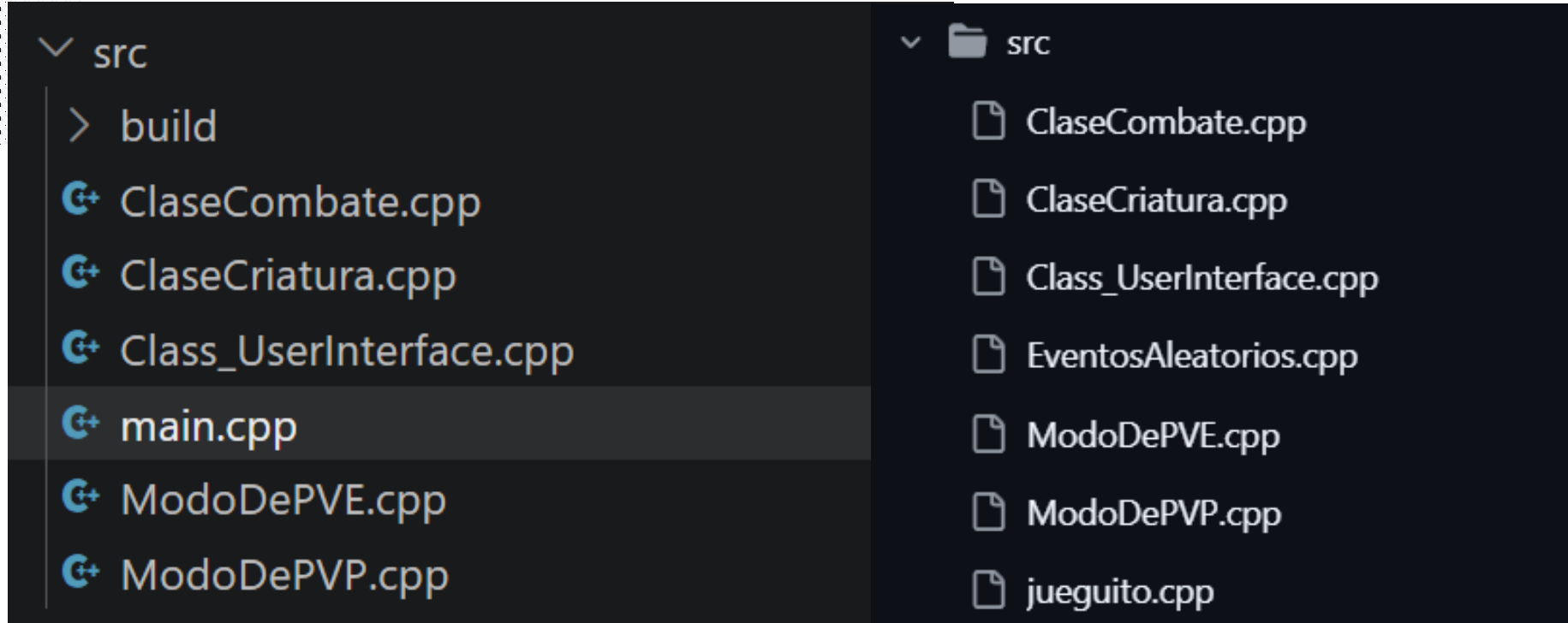
REORGANIZACIÓN V0.1.1

```
> docs
  < lib
    < ModoBatalla
      + ClaseCombate.hpp
      + ClaseCriatura.hpp
      + ModoDePVE.hpp
      + ModoDePVP.hpp
    < ModoHistoria
      + Class_UserInterface.hpp
```

```
  < lib
    < ModoCombate
      ClaseCombate.hpp
      ClaseCriatura.hpp
      ModoDePVE.hpp
      ModoDePVP.hpp
    < ModoHistoria
      Class_UserInterface.hpp
```

Implementación: Derek

REORGANIZACIÓN V0.1.1



Implementación: Derek

CÓDIGO V0.1.1

```
1  /* LIBRERIAS STANDARD / EXTERNAS */
2  #include <iostream>
3  #include <string>
4
5  /* LIBRERIAS PROPIAS */
6  /*
7      En este apartado, iran todas las librerias que se vayan implementando conforme avance el desarrollo del juego.
8      Un ejemplo de esto es la clase Class_UserInterface.hpp, que se movio a un archivo a parte para limpiar el main.
9      Esto se debera hacer con cada una de las demas funciones agregadas al proyecto para mantener un orden dentro del proyecto.
10     Si necesitan agregar librerias externas, agreguenlas directamente a la parte superior junto a <iostream> y las demas.
11 */
12 #include "../lib/ModoHistoria/Class_UserInterface.hpp"
13 #include "../lib/ModoBatalla/ClaseCombate.hpp"
14 #include "../lib/ModoBatalla/ClaseCriatura.hpp"
15 #include "../lib/ModoBatalla/ModoDePVE.hpp"
16 #include "../lib/ModoBatalla/ModoDePVP.hpp"
17
```

Implementación: Derek

CÓDIGO V0.1.1

```
27  ✓ int main(){  
28  
29      /* Se crea la interfaz de usuario UI */  
30      std::string texto = "Bienvenido al juego.";  
31      bool Termino = false; //ESTA VARIABLE SOLO EXISTE  
32      ModoDeJuegoPVE PVE;  
33      ModoDeJuegoPVP PVP;
```

Implementación: Derek

CÓDIGO V0.1.1

```
57      // EJEMPLO DE IMPLEMENTACION DE LOS MODOS DE JUEGO
58      switch(UI.GetOption()){
59          case 1:
60
61              PVP.CursoDeCombatePVP();
62              break;
63
64          case 2:
65
66              PVE.CursoDeBatallaPVE();
67              break;
68
69          default: break;
```

Implementación: Derek

EJECUCIÓN V0.1.1

```
== BIENVENIDO A JUEGUITO ==
```

```
1: Modo Historia
```

```
2: Modo Batalla
```

```
0: Salir
```

```
-> 2
```

```
1: Modalidad PvP
```

```
2: Modalidad PvE [BOT]
```

```
3: Regresar
```

```
-> █
```

Implementación: Derek

TESTEO

Durante el desarrollo de todo el proyecto, sólo se ha realizado un testeo a una versión final:

V0.0.1.1: Verificación de Errores en las funciones de la clase *UserInterface*.

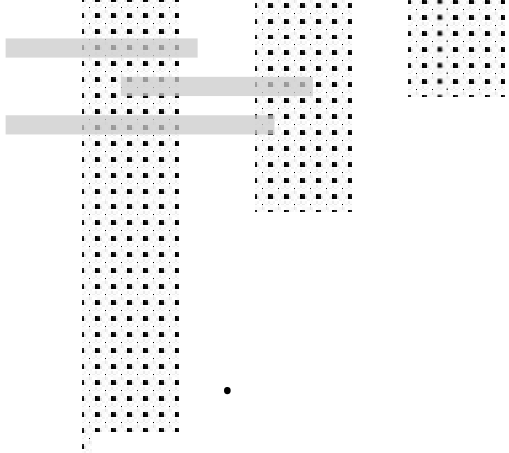
En las demás versiones, no se ha podido realizar el testeo correspondiente debido a limitaciones de tiempo. Los responsables del testeo fueron Luis y Javier, quienes realizaron un documento explicando lo encontrado.

TESTEO V0.0.1.1

El Error se encontró dentro de la función Input:

```
/* Se pide la opcion a escoger */
std::cin >> Option;

/* Si el usuario mete alguna letra o numero indeseado, se ejecutara el "if" para
if(std::cin.fail() || std::cin.peek() != '\n'){
    std::cin.clear();
    std::cin.ignore(100 /* Rango de limpieza de 100 caracteres */, '\n');
    Option = InferiorLimit-1;
}
```



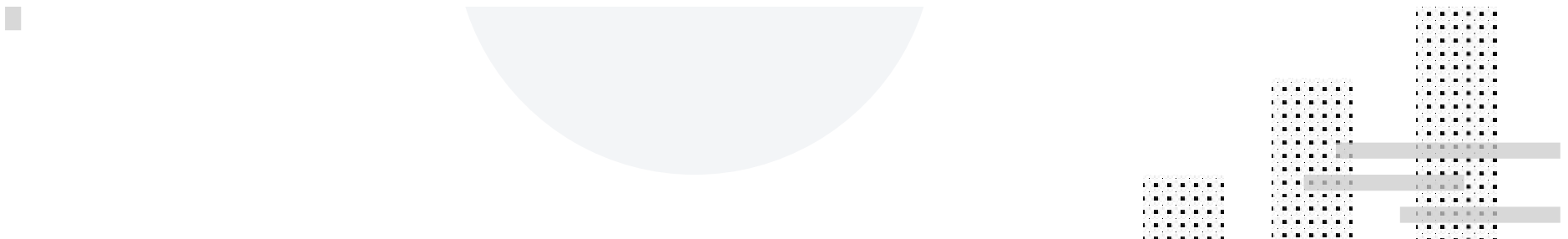
TESTEO V0.0.1.1

•

BUG-001

Descripción:

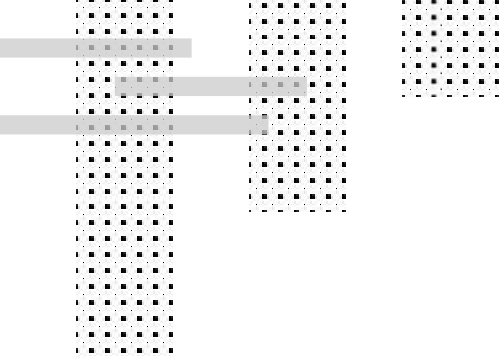
Al ingresar una cadena de texto que supera los 100 caracteres antes de presionar Enter, el programa no logra limpiar completamente la memoria de entrada (std::cin). Los caracteres restantes quedan almacenados y son procesados automáticamente por la siguiente lectura, provocando bucles no deseados en la interfaz y saltándose la interacción con el usuario.



TESTEO V0.0.1.1

Pasos para Reproducir:

1. *Ejecutar el programa compilado.*
2. *En la primera solicitud de entrada (Elije una opción para continuar), ingresar una secuencia de texto de más de 100 caracteres seguida de un espacio y un número dentro del rango (ejemplo:
aa
aa 2).*
3. *Presionar Enter.*



TESTEO V0.0.1.1

Resultado Obtenido (Error):

El programa realiza múltiples repeticiones de la instrucción en consola sin pausar para pedir datos al usuario. Lee automáticamente el valor 2 que quedó al final del búfer residual, valida la opción e interrumpe el flujo normal de la prueba.

Resultado Esperado:

El programa debe ignorar la totalidad de la entrada inválida, mostrar una sola vez el mensaje de reintento y quedar a la espera de que el usuario vuelva a escribir una opción por teclado.



DOCUMENTACIÓN

Al igual que en el caso de el Testeo, hasta ahora por limitaciones de tiempo sólo se ha podido hacer la documentación de las versiones:

V0.0.1: Explicando lo nuevo

V0.0.1.1: Explicando lo nuevo, el testeo y la funcionalidad total

Esto es, en sí, un resumen de lo hecho. El responsable de la documentación fue Christian.

DOCUMENTACIÓN

01

Ver si hay algo nuevo
en el repositorio

03

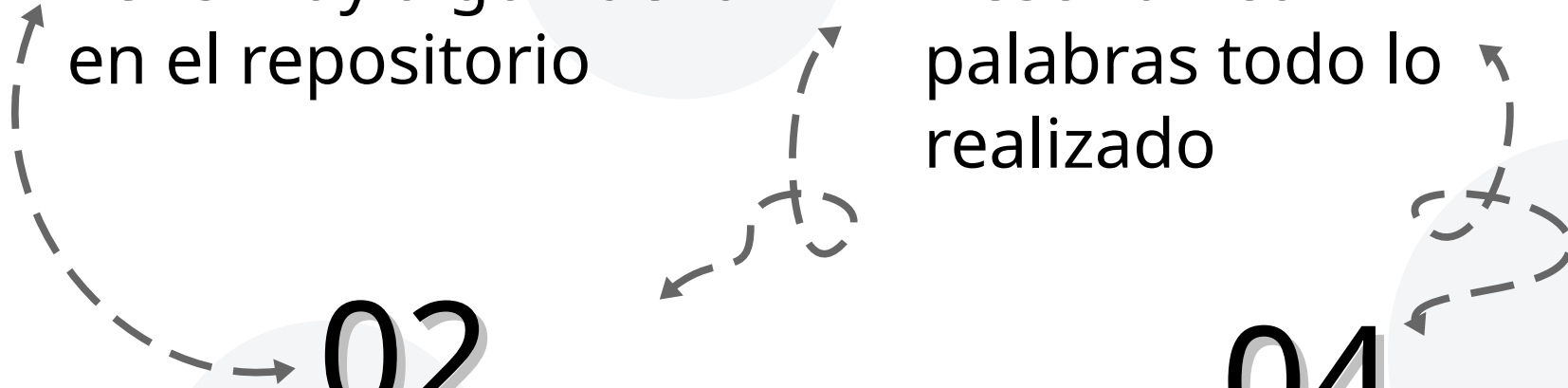
Describir con
palabras todo lo
realizado

02

Analizar todos los
cambios hechos en
el mismo

04

Subir la
documentación al
repositorio





DOCUMENTACIÓN V0.0.1.1

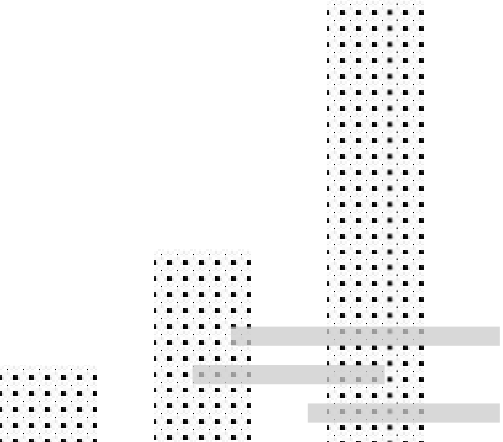
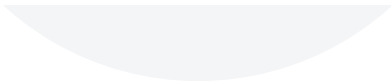
UNIVERSIDAD DE GUADALAJARA
Centro Universitario de los Valles

Reporte de desarrollo y pruebas del proyecto JUEGUITO

Interfaz de entrada y salida de texto en C++
Versiones documentadas: 0.0.1 y 0.0.1.1

Equipo: Los Cholos
Grupo: 3A Mecatrónica

Programación Orientada a Objetos · 2026B
Mtro. Amed Aldama Bernabeu



DOCUMENTACIÓN V0.0.1.1

2. Objetivos

Objetivo general. Analizar el desarrollo y la validación de una interfaz de usuario en C++ que permita mostrar texto y recibir únicamente opciones enteras dentro de un rango definido.

Objetivos específicos. Identificar la función de los métodos de entrada y salida; explicar su organización en una clase; documentar el error informado por el tester; y contrastar ese reporte con el comportamiento del código adjunto.

3. Evolución del proyecto

Versión 0.0.1, 3 de septiembre de 2026. La propuesta establece las funciones Output() e Input(). La primera muestra una cadena de texto; la segunda recibe un límite inferior, un límite superior y una pregunta. La entrada debe repetirse hasta obtener un número permitido.

Versión 0.0.1.1, 11 de septiembre de 2026. Se propone integrar ambas funciones como métodos de una clase llamada InterfazUsuario. Este cambio organiza las responsabilidades de presentación y lectura dentro de un objeto reusable.

Implementación entregada. El código emplea el nombre UserInterface, conserva la opción elegida en un atributo privado y agrega GetOption() para consultarla. A diferencia de la especificación, Input() está declarado como void: almacena el resultado, pero no lo devuelve directamente.

DOCUMENTACIÓN V0.0.1.1

4. Aplicación de programación orientada a objetos

- La clase encapsula el atributo Option y expone métodos públicos para interactuar con él. Su constructor inicializa ese atributo en cero. La instrucción `UserInterface UI;` crea un objeto de la clase; técnicamente se trata de instanciación, no de herencia. El prototipo evidencia encapsulamiento y uso de objetos, sin implementar herencia ni polimorfismo.

5. Funcionamiento del código proporcionado

Elemento	Responsabilidad
Option	Atributo privado de tipo <code>int</code> que almacena la opción.
<code>UserInterface()</code>	Constructor que inicializa Option en 0.
<code>Output(const std::string& Text)</code>	Imprime el texto y un salto de línea.
<code>Input(int, int, const std::string&)</code>	Muestra la pregunta y las opciones; lee y valida la entrada.
<code>GetOption()</code>	Devuelve el valor almacenado en Option.

DOCUMENTACIÓN V0.0.1.1

6. Incidencia reportada por el tester: BUG-001

Descripción del reporte. El tester señala que una cadena de más de 100 caracteres, seguida de un espacio y un número válido, deja datos residuales en el búfer. Según su evidencia, estos datos provocan repeticiones automáticas y la aceptación de un número sin una nueva interacción.

Procedimiento descrito. Ejecutar el programa; escribir más de 100 letras en la primera pregunta; agregar un espacio y el número 2; y presionar Enter.

Resultado esperado. Descartar completamente la línea inválida, avisar al usuario y esperar una nueva opción. El número ubicado al final de esa misma línea no debe interpretarse como una respuesta independiente.

Impacto. Si ocurre, el fallo afecta el control del usuario sobre las decisiones y la confiabilidad de los menús. Por ello es necesario comprobar tanto la validación del número como el consumo completo de la entrada.

TABLA DE DIVISIÓN DE ROLES

ROLES	V0.0.1	V0.0.1.1	V0.1.0	V0.1.1
Arquitecto	Derek	Derek	Derek	Derek
Programador	Erick, Omar, Derek	Derek	Javier, Luis, Derek	Javier, Luis, Omar, Derek
Tester	Javier, Luis	Javier, Luis	No Hecho	No Hecho
Documentador	Christian	Christian	No Creado	No Creado

FINALMENTE...

MUCHÍSIMAS GRACIAS POR ESCUCHARNOS!

Integrantes de ***Los Cholos***:

Derek Salcido González

Omar Contreras Paz

Javier Mora Gutierrez

Luis Fernando Duarte Zárate

Erick Yael Ramos Ramirez

Christian Osvaldo Guzman Orozco

